B.Tech CSE (II YEAR – IV SEM) (2025-2026)

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING & APPLICATIONS

GLA UNIVERSITY

17km Stone, NH-19, Mathura-Delhi Road, P.O. Chaumuhan, Mathura – 281406 (Uttar Pradesh) India

Project Title:

EVERWRITE – Interactive Storytelling and Gaming Using Adaptive Generative AI

Team Leader:	Kushagra Agrawal	UR:2415500261
Team Member 1:	Utkarsh Agrawal	UR:2415500490
Mentor Name:	**Mr. Shivanshu Upadhyay**	

Signature:

1. Abstract

Interactive storytelling has emerged as a significant area in the fields of digital entertainment, artificial intelligence, and educational technology. Traditional narrative systems rely heavily on predefined story structures that restrict creativity and limit the number of possible outcomes. These systems require extensive manual development of story branches, making them time-consuming and difficult to scale.

EverWrite is an AI-powered interactive storytelling platform designed to generate dynamic narratives based on user input. The system uses Large Language Models (LLMs) to create immersive story experiences that evolve according to the user's decisions.

The platform integrates the Gemini API for narrative generation, a Flask backend for managing game logic and API communication, and a vector-based memory system to maintain narrative continuity. The frontend interface provides a retro-themed interactive storytelling environment and communicates with the backend using real-time communication techniques.

By combining generative AI, vector databases, and modern web technologies, EverWrite demonstrates how AI can transform storytelling into a dynamic and personalized experience.

---

2. Introduction

Storytelling has always been an important medium for communication, education, and entertainment. With the rise of digital technology, storytelling has evolved into interactive forms where users can actively influence the progression of narratives.

Traditional interactive storytelling systems often rely on branching narratives where developers manually design multiple story paths. Although this approach allows some level of interactivity, it limits the number of possible outcomes and requires a large amount of manual effort.

Recent advancements in artificial intelligence have enabled systems to generate text dynamically using large-scale neural networks known as Large Language Models. These models can produce coherent and context-aware narratives based on prompts and contextual inputs.

EverWrite utilizes these advancements to create an AI-driven storytelling platform that dynamically generates story content in response to user actions. The system uses a modular architecture consisting of a frontend interface, backend game engine, AI integration module, and memory storage system.

This approach significantly increases replayability, improves narrative flexibility, and reduces the need for manual story creation.

---

3. Problem Statement

Traditional storytelling games suffer from several limitations:

- Limited number of narrative outcomes
- High development effort required to create branching storylines
- Inability to respond to unique user inputs
- Reduced replayability once story paths are explored

These limitations highlight the need for an intelligent storytelling system capable of generating narrative content dynamically. Instead of relying on predefined scripts, the system should be able to adapt to user actions and create new story content in real time.

The EverWrite project aims to solve this problem by integrating generative AI with a game engine capable of maintaining narrative context and dynamically generating story responses.

---

4. Objectives

The main objectives of this project are:

- To design and develop an AI-powered storytelling platform
- To integrate a Large Language Model for dynamic narrative generation
- To implement a vector memory system for contextual recall
- To create a responsive and user-friendly web interface
- To enable real-time interaction between the user and the AI system
- To maintain narrative consistency across multiple interactions

These objectives ensure the creation of a scalable and engaging storytelling system.

---

5. Literature Review

Early interactive storytelling systems relied on rule-based models and branching narrative trees. These systems required developers to manually write every possible storyline, which limited scalability.

With the advancement of machine learning and deep learning technologies, transformer-based models such as GPT and Gemini have demonstrated remarkable capabilities in natural language generation.

Vector databases have also become increasingly important in AI applications. They allow systems to store textual information as embeddings and retrieve relevant context using similarity search techniques.

By combining these technologies, EverWrite creates a system capable of generating dynamic narratives while maintaining contextual awareness through memory retrieval.

---

6. Team Roles and Contributions

The EverWrite project was developed collaboratively by team members with clearly defined responsibilities.

S.No	Team Member	Role	Responsibilities
1	Utkarsh Agrawal	Project Lead / Backend Developer	Designed system architecture, implemented Flask APIs, integrated Gemini API
2	Utkarsh Agrawal	Frontend Developer	Developed UI using HTML, CSS, and JavaScript
3	Kushagra Agrawal/ Utkarsh Agrawal	AI Integration Engineer	Implemented prompt engineering and AI interaction
4	Kushagra Agrawal	Memory System Developer	Implemented vector database and embeddings
5	Kushagra Agrawal	Testing & Documentation	Performed system testing and prepared documentation

---

7. Technology Stack

The EverWrite system utilizes multiple technologies across different layers.

Layer	Technology Used	Purpose
Frontend	HTML	Structure of the user interface
Frontend	CSS	Styling and layout
Frontend	JavaScript	Interactive functionality
Backend	Python	Core backend programming
Backend	Flask	Web server and API framework
AI Model	Gemini API	Narrative text generation
Memory	ChromaDB	Vector database storage
Embeddings	Sentence Transformers	Text embedding generation
Communication	SSE	Real-time data streaming

---

8. System Architecture

1. Frontend Layer

This is the **Presentation Layer**. Its primary job is to provide a seamless, immersive experience while shielding the user from the technical "gears" turning in the background.

- **Narrative Rendering:** Handles the aesthetic delivery of the story (typewriter effects, scrolling history, and font styling).
- **Input Capture:** Collects user text or intent (e.g., "Open the chest") and sanitizes it before sending it to the Backend.
- **Local UI State:** Manages visual-only elements like character portraits, health bars, or inventory icons based on data received from the Backend.

2. Backend Layer

This is the **Logic and Orchestration Layer**. It acts as the "referee" and the central hub that connects the user to the AI and the database.

- **Game Engine & Rules:** Processes player actions against the current game logic (e.g., checking if an item is in the inventory).
- **State Management:** Holds the "Single Source of Truth." It tracks every variable, from the player's gold to whether a specific NPC is angry.
- **Request Routing:** Coordinates the flow of information, deciding when to pull from Memory and when to send a prompt to the AI.

3. AI Layer

This is the **Intelligence Layer**. It focuses specifically on the creative synthesis of the narrative.

- **Prompt Construction:** Merges the Backend's game state with "System Instructions" to guide the LLM's personality and boundaries.

- **Gemini API Integration:** Manages the actual communication with the model, including handling tokens and streaming responses back for a faster "feel."
- **Narrative Filtering:** Ensures the AI's output stays within the game's world-building constraints before passing it back to the user.

4. Memory Layer

This is the **Persistence and Context Layer**. It solves the "Goldfish Memory" problem common in standard LLM chats.

- **Vector Database:** Stores past story beats as high-dimensional vectors (embeddings) rather than just raw text.
- **Contextual Retrieval:** When a player mentions a character from three hours ago, this layer "remembers" the relevant snippets and feeds them into the current AI prompt.
- **Long-Term Archiving:** Keeps a permanent record of the entire playthrough history for later analysis or session reloading.

---

9. Project Directory Structure

```
EverWrite/
├── backend/
│   ├── app.py
│   ├── config.py
│   ├── main.py
│   ├── game/
│   │   ├── engine.py
│   │   ├── prompt.py
│   │   └── state.py
│   ├── llm/
│   │   └── gemini.py
│   └── memory/
│       └── vector_store.py
├── frontend/
│   ├── static/
│   │   ├── css/style.css
│   │   └── js/game.js
│   └── templates/index.html
├── run.py
└── requirements.txt
```

---

10. System Modules

1. Game Engine

The "Brain" of the application. It acts as the central orchestrator that coordinates the flow of data between all other modules.

- **Logic Loops:** Decides when to trigger a narrative beat versus when to wait for player input.
- **Rule Enforcement:** Ensures player actions are physically or logically possible within the game's "hard rules" (e.g., you can't cast a spell if your mana is zero).
- **Event Dispatching:** Sends updates to the **Frontend** and signals the **State Manager** to save progress after every turn.

2. Prompt Builder

The "Translator" that turns raw data into something the AI can understand. This is where the "flavor" of your game is born.

- **Context Assembling:** Grabs the current location, active NPCs, and recent history to create a rich prefix for the LLM.
- **Dynamic Templating:** Injects specific instructions like *"Write in a noir style"* or *"Limit response to two paragraphs."*
- **Persona Management:** Adjusts the prompt based on who the player is talking to, ensuring the AI adopts the correct tone and knowledge set.

3. State Manager

The "Source of Truth" for the current session. While the AI handles the prose, the State Manager handles the numbers and facts.

- **Variable Tracking:** Manages inventory, health, relationship scores, and "World Flags" (e.g., `is_dragon_dead = true`).

- **Session Persistence:** Handles saving and loading game files so players can resume their journey.
- **Snapshotting:** Keeps a record of the immediate "short-term memory" to prevent the story from looping or forgetting what happened ten seconds ago.

4. LLM Integration

The "Gateway" to the Gemini API. This layer abstracts the technical complexities of AI communication.

- **API Management:** Handles authentication, rate limiting, and error handling (e.g., retrying if the connection drops).
- **Hyperparameter Tuning:** Controls settings like `temperature` (randomness) and `top_p` to ensure the story is creative but not nonsensical.
- **Output Parsing:** Cleans up the AI's response, stripping away unwanted "AI chatter" or formatting it into JSON for the Game Engine to read.

5. Vector Memory

The "Long-Term Memory." This allows the game to remember details from 10 chapters ago without overloading the prompt.

- **Embedding Generation:** Converts every story beat into a numerical vector (a mathematical "fingerprint" of the meaning).
- **Semantic Retrieval:** When a player mentions an old character, the system searches the database for the most *relevant* past events, not just keyword matches.
- **Context Pruning:** Feeds only the most important historical facts back to the **Prompt Builder** to stay within the AI's token limits.

6. Frontend Interface

The "Window" into the world. This is the user-facing layer where the magic happens.

- **Narrative Delivery:** Renders the AI's text, often with typewriter effects or visual cues to enhance immersion.
- **Input Handling:** Captures player text or button choices and passes them to the **Game Engine**.
- **Visual/Audio Feedback:** Displays character portraits, inventory UI, or background music that changes based on the "State" (e.g., playing combat music when a flag is triggered).

---

11. API Endpoints

The backend provides RESTful API endpoints to handle communication between the frontend and backend.

Endpoint	Method	Description
/api/start	POST	Initializes a new game session
/api/chat	POST	Sends player action and receives AI-generated narrative
/api/reset	POST	Resets the current game session
/api/status	GET	Returns current game state

---

12. Implementation

The backend is implemented using Flask, which provides a lightweight framework for building APIs.

The game engine processes each player action by:

1. Retrieving relevant narrative context from the memory store
2. Constructing a prompt using the prompt builder
3. Sending the prompt to the Gemini API

4. Receiving and returning the generated narrative

The vector memory system stores important story events as embeddings and retrieves similar events when generating new narrative responses.

---

13. Frontend Design

The frontend interface is designed with a retro terminal aesthetic to enhance immersion.

The main components include:

- Narrative display panel
- User input box
- Game status indicators

JavaScript handles communication with the backend using asynchronous HTTP requests and real-time streaming mechanisms.

---

14. Testing

Several testing methods were used to evaluate the system.

Functional Testing

Ensures that all API endpoints operate correctly.

Integration Testing

Verifies communication between frontend, backend, and AI services.

Performance Testing

Measures system response time and memory retrieval efficiency.

User Testing

Evaluates usability and storytelling quality.

---

15. Results and Discussion

The EverWrite system successfully generates dynamic narratives that respond to user decisions.

Key results include:

- Unique story outcomes for each playthrough
- Improved narrative coherence through memory retrieval
- Real-time response streaming for improved user experience

These results demonstrate the effectiveness of integrating generative AI with vector memory.

---

16. Applications

EverWrite can be used in various domains:

Interactive Gaming

Dynamic narrative games with unlimited replayability.

Education

Scenario-based learning platforms.

Creative Writing

Story idea generation for writers.

Conversational AI

Narrative-driven chatbot experiences.

---

17. Project Timeline

Phase	Activities	Duration
Phase 1	Idea Selection & Research	Week 1
Phase 2	System Design & Architecture	Week 2

Phase 3	Backend Development	Week 3–4
Phase 4	AI Integration	Week 5
Phase 5	Frontend Development	Week 6
Phase 6	Testing & Debugging	Week 7
Phase 7	Documentation & Finalization	Week 8

---

18. Future Work

Future improvements may include:

- Multiplayer storytelling mode
- Voice-based interaction with AI
- Improved memory retrieval algorithms
- Fine-tuned narrative models
- Mobile application support

---

19. Conclusion

EverWrite demonstrates the potential of combining generative AI with modern web technologies to create dynamic storytelling systems.

The integration of Large Language Models with vector memory and a modular game engine allows the platform to generate coherent narratives that evolve according to user decisions.

This project highlights the growing role of artificial intelligence in creative applications and provides a foundation for future development of intelligent storytelling platforms.